What if we keep the attention and drop the recurrence?

$$X \to \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \to \mathrm{block}^{\times D} \to p_\theta(w_{t+1} \mid w_{\le t}) \to \mathrm{adapt}$$

Roadmap

1. Self-attention in full — heads, masks, position, cost
2. The Transformer block — attention plus everything from Session 02
3. Pretraining — tokens, next-token prediction, masked prediction
4. Scale and foundation models — text, images, time series, tables
5. Using a pretrained model — zero-shot, frozen, fine-tuned, LoRA

Practice A after Part 2; Practice B after Part 5.

Where Session 05 left us

Sequence to sequence with attention: at every output step the decoder **reads every encoder state** directly.

But the encoder is still an RNN:

- step t waits for step $t-1$: training does not parallelize along T;
- inside the encoder, two distant words are still T steps apart.

The question for today

> **What if we keep the attention and drop the recurrence?**

- If every token can read every other token, is the recurrence needed at all?
- And once we have such a model, what happens when we train it on a lot of text?

Part 1 · Self-attention in full

1. **Self-attention in full** — heads, masks, position, cost
2. The Transformer block — attention plus everything from Session 02
3. Pretraining — tokens, next-token prediction, masked prediction
4. Scale and foundation models — text, images, time series, tables
5. Using a pretrained model — zero-shot, frozen, fine-tuned, LoRA

> Session 05: attention was a lookup from the **decoder** into the **encoder**. Today every token looks into its **own** sequence.

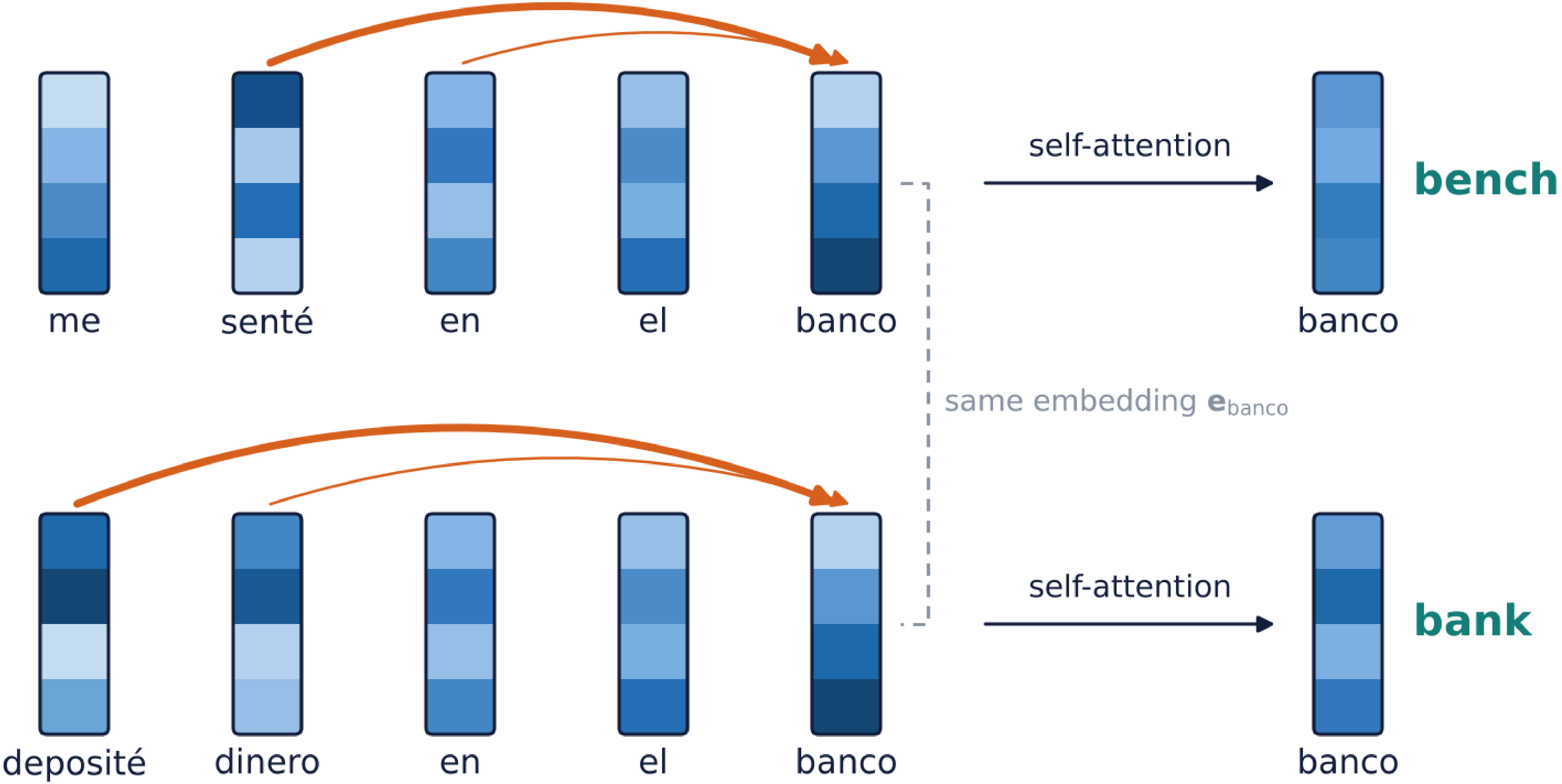
From cross-attention to self-attention

Session 05: the decoder state $\mathbf{s}_{\tau-1}$ asked; the encoder states $\mathbf{h}_t$ answered.

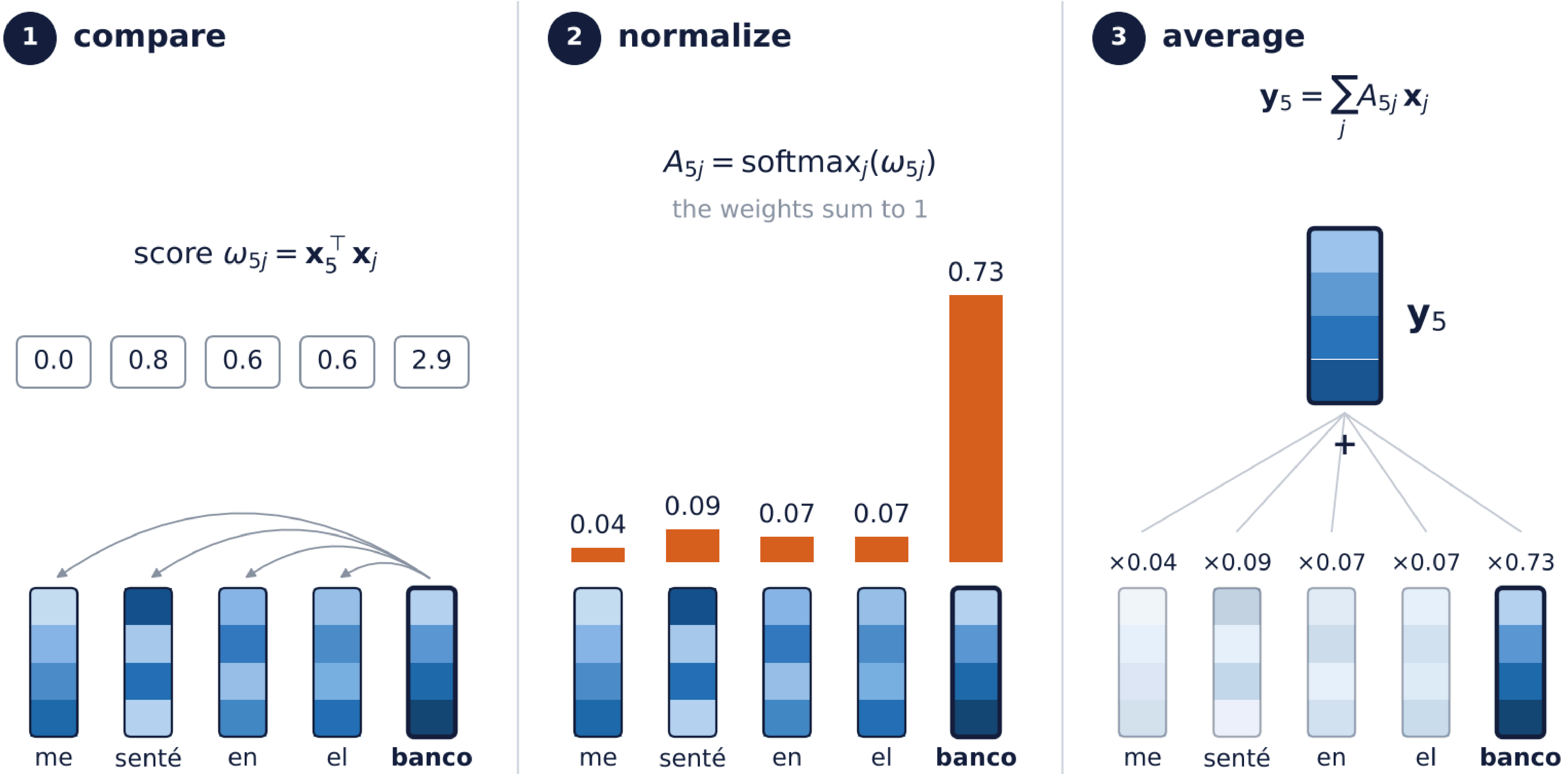
Now: every token asks **every token of the same sequence**, itself included. The simplest version, with no parameters at all:

$$\omega_{ij} = \mathbf{x}_i^\top \mathbf{x}_j, \qquad A_{ij} = \mathrm{softmax}_j(\omega_{ij}), \qquad \mathbf{y}_i = \sum_{j=1}^{T} A_{ij}\, \mathbf{x}_j$$

Compare, normalize, average. The weights depend on the **content** of the tokens, not on a fixed neighborhood (Session 03).


me
senté
en
el
banco
self-attention
banco
bench
same embedding $\mathbf{e}_{\text{banco}}$
deposité
dinero
en
el
banco
self-attention
banco
bank

the input vector is the same; the output depends on the other tokens



1 compare

score $\omega_{5j} = \mathbf{x}_5^\top \mathbf{x}_j$

0.0
0.8
0.6
0.6
2.9
me
senté
en
el
banco

2 normalize

$A_{5j} = \mathrm{softmax}_j(\omega_{5j})$

the weights sum to 1

0.04
0.09
0.07
0.07
0.73
me
senté
en
el
banco

3 average

$\mathbf{y}_5 = \sum_j A_{5j}\,\mathbf{x}_j$

y5
+
×0.04
×0.09
×0.07
×0.07
×0.73
me
senté
en
el
banco

no parameters: every token is most similar to itself, so $\mathbf{y}_5$ is almost $\mathbf{x}_5$

Query, key, value

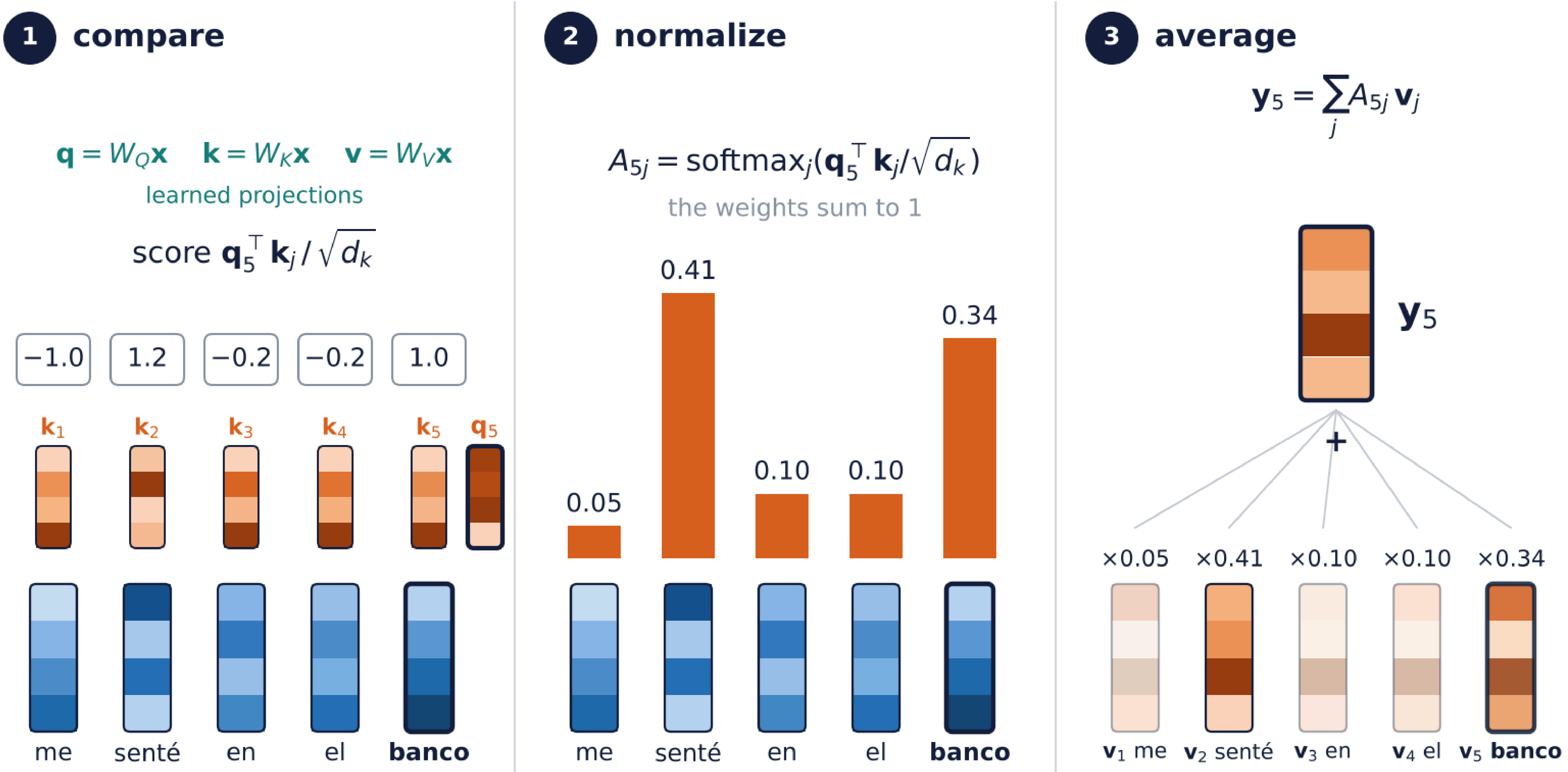
Without parameters, $\mathbf{x}_i^\top \mathbf{x}_i$ is usually the largest score: each token mostly reads **itself**, and the similarity is the same for every task.

Give each token three roles, each with its own **learned** projection:

Role	Matrix	Question it answers
query	$Q = XW_Q$	what am I looking for?
key	$K = XW_K$	what do I offer to be found by?
value	$V = XW_V$	what do I hand over if I am chosen?

Row i of X is token $\mathbf{x}_i$, as in Session 03.

A **soft dictionary lookup**: compare the query with every key, return a mixture of the values.



①
compare
②
normalize
③
average
−1.0
1.2
−0.2
−0.2
1.0
$\mathbf{k}_1$
$\mathbf{k}_2$
$\mathbf{k}_3$
$\mathbf{k}_4$
$\mathbf{k}_5$
$\mathbf{q}_5$
me
senté
en
el
banco
0.05
0.41
0.10
0.10
0.34
me
senté
en
el
banco
$\mathbf{y}_5$
+
×0.05
×0.41
×0.10
×0.10
×0.34
$\mathbf{v}_1$ me
$\mathbf{v}_2$ senté
$\mathbf{v}_3$ en
$\mathbf{v}_4$ el
$\mathbf{v}_5$ banco

$$\mathbf{q} = W_Q\mathbf{x} \quad \mathbf{k} = W_K\mathbf{x} \quad \mathbf{v} = W_V\mathbf{x}$$

learned projections

score $\mathbf{q}_5^\top \mathbf{k}_j / \sqrt{d_k}$

$$A_{5j} = \mathrm{softmax}_j(\mathbf{q}_5^\top \mathbf{k}_j/\sqrt{d_k})$$

the weights sum to 1

$$\mathbf{y}_5 = \sum_j A_{5j}\,\mathbf{v}_j$$

learned projections: "banco" now looks mostly at "senté"

Scaled dot-product attention

$$\mathrm{Attention}(Q,K,V) = \underbrace{\mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)}_{A \in \mathbb{R}^{T\times T}} V$$

- $S = QK^\top/\sqrt{d_k}$: all T^2 scores in **one matrix product**;
- softmax **row by row**: row i is a distribution over the tokens that i reads;
- the dot-product score of Session 05, now for every pair at once.

```
S = Q @ K.transpose(-2, -1) / math.sqrt(d_k)   # (T, T)
A = S.softmax(dim=-1)                           # each row sums to 1
Y = A @ V                                       # (T, d_k)
```

Self-attention in motion

S06-A01 · Self-attention

A 5-token sentence. Each token emits $\mathbf{q}, \mathbf{k}, \mathbf{v}$; token 3 compares its $\mathbf{q}$ with every $\mathbf{k}$, the scores become a softmax row, the row averages the $\mathbf{v}$'s. Then all rows at once as $QK^\top$, and the causal mask greys out the upper triangle.

`../figures/s06_a01_self_attention.mp4` · route: Manim (`escenas_s06.py`)

Why divide by $\sqrt{d_k}$?

If the entries of $\mathbf{q}$ and $\mathbf{k}$ are independent with mean 0 and variance 1:

$$\mathrm{Var}\left(\mathbf{q}^\top \mathbf{k}\right) = \sum_{m=1}^{d_k} \mathrm{Var}(q_m k_m) = d_k$$

With $d_k = 64$ the scores have standard deviation 8: the softmax **saturates**, one weight near 1, the rest near 0, and its gradient vanishes.

Dividing by $\sqrt{d_k}$ brings the variance back to 1. The same variance argument as **initialization** in Session 02.

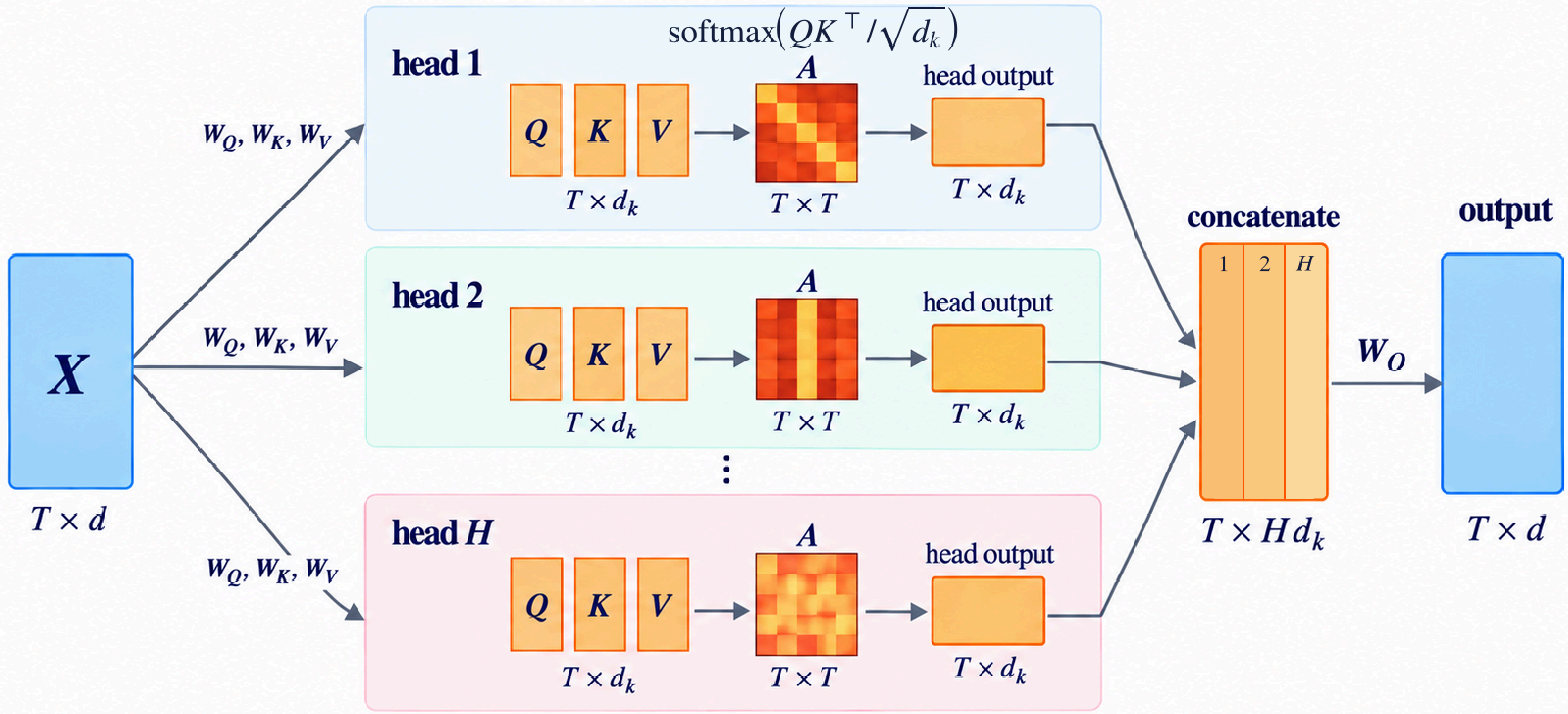
Multi-head attention

One head computes **one** pattern of who reads whom. Use H heads of width $d_k = d/H$:

$$\text{head}_h = \text{Attention}\left(XW_Q^{(h)}, XW_K^{(h)}, XW_V^{(h)}\right), \qquad \text{MHA}(X) = \left[\,\text{head}_1, \ldots, \text{head}_H\,\right] W_O$$

- each head can attend to a different relation (the next word, the subject, a closing bracket);
- **same parameters and cost** as one head of width d: $4d^2$ in total;
- like several kernels in one convolutional layer (Session 03).

In code, heads are a reshape: `(B, T, d) → (B, H, T, d_k)`.


X
$T \times d$
W_Q, W_K, W_V
W_Q, W_K, W_V
W_Q, W_K, W_V
$\mathrm{softmax}\left(QK^\top/\sqrt{d_k}\right)$
head 1
Q K V
$T \times d_k$
A
$T \times T$
head output
$T \times d_k$
head 2
Q K V
$T \times d_k$
A
$T \times T$
head output
$T \times d_k$
⋮
head H
Q K V
$T \times d_k$
A
$T \times T$
head output
$T \times d_k$
concatenate
1 2 H
$T \times H d_k$
W_O
output
$T \times d$

$d_k = d\,/\,H$: same parameters and cost as one head of width d

Masks

$$A = \mathrm{softmax}(S + M), \qquad M_{ij} = \begin{cases} 0 & \text{if } i \text{ may read } j \\ -\infty & \text{otherwise} \end{cases}$$

$e^{-\infty} = 0$: a masked position gets weight exactly 0, and the row still sums to 1.

- **Padding mask:** no token reads `[PAD]` (Session 05: batches of different lengths).
- **Causal mask:** token i reads only $j \le i$. The model cannot see the future, which is what a generator needs.

```
causal = torch.ones(T, T).triu(diagonal=1).bool()   # True above the diagonal
S = S.masked_fill(causal, float('-inf'))
```

Prediction before experiment

A self-attention layer **without** positional information. Feed it a sentence, then the same tokens in a shuffled order.

No mask: what happens to the outputs when the inputs are shuffled?

Causal mask: and now?

Answer in notebook 6.1.

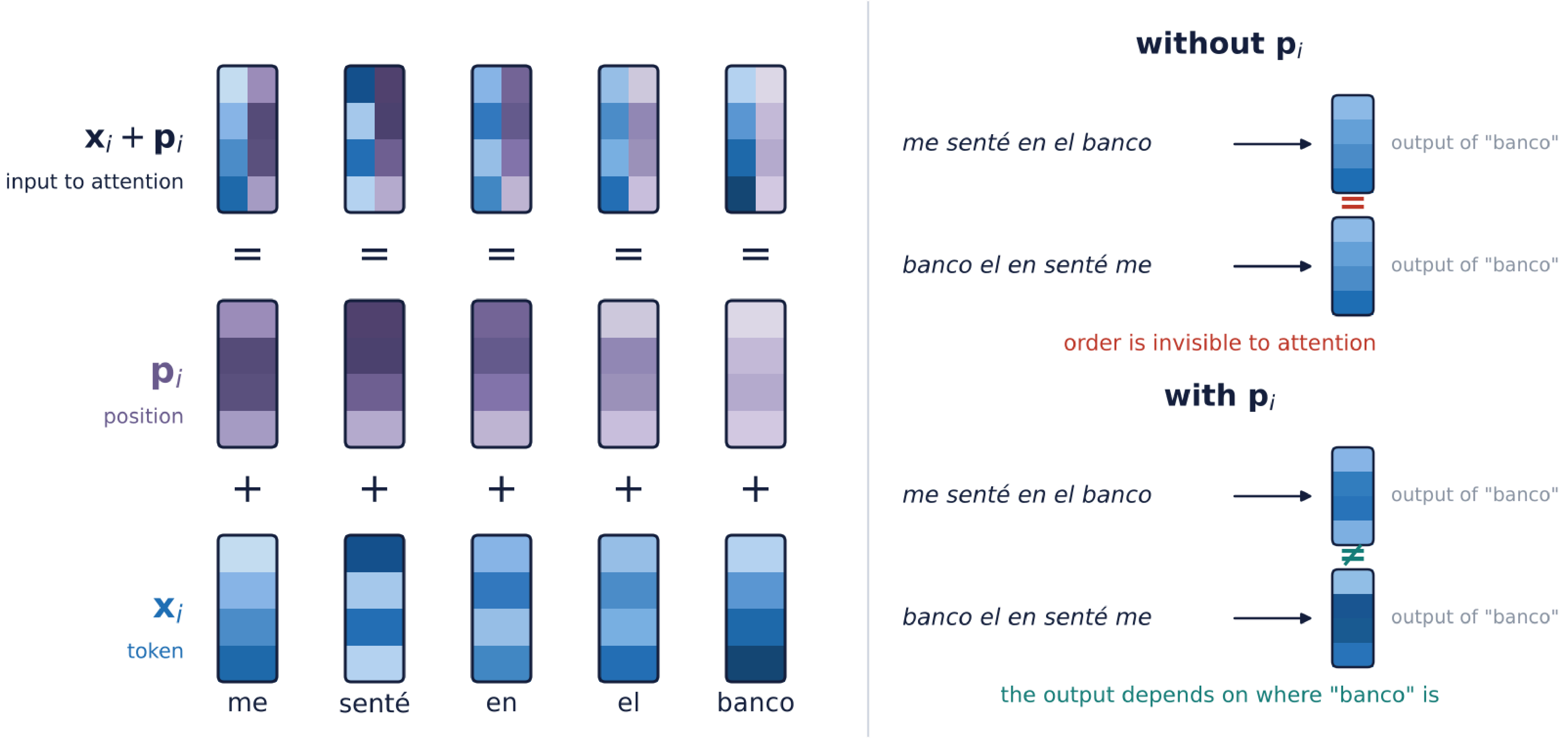
Order has to be added

Without position, self-attention is **permutation-equivariant**: shuffle the inputs and the outputs are shuffled the same way.

Session 03 (patches) and Session 05 (shuffled words) showed why that matters.

Add a vector $\mathbf{p}_i$ for each position: $\mathbf{x}_i \leftarrow \mathbf{x}_i + \mathbf{p}_i$

	How	**Where**
learned	one trained vector per position	ViT (Session 03), BERT, GPT-2
sinusoidal	fixed sines and cosines of different frequencies	the original Transformer (Vaswani et al., 2017)
rotary (RoPE)	rotate $\mathbf{q}$ and $\mathbf{k}$ by an angle that depends on the position	most current LLMs


$\mathbf{x}_i + \mathbf{p}_i$
input to attention
=
$\mathbf{p}_i$
position
+
$\mathbf{x}_i$
token
me
senté
en
el
banco
without $\mathbf{p}_i$
me senté en el banco
output of "banco"
=
banco el en senté me
output of "banco"
order is invisible to attention
with $\mathbf{p}_i$
me senté en el banco
output of "banco"
≠
banco el en senté me
output of "banco"
the output depends on where "banco" is

self-attention sees a set of tokens; adding $\mathbf{p}_i$ turns it into a sequence

Masks and position, measured

S06-F06 · Masks and permutation

Attention matrices of one sentence with no mask, with a padding mask and with a causal mask; then the outputs before and after shuffling the tokens, without positional embeddings.

`../figures/s06_f06_masks_permutation.png` · route: Script (notebook 6.1)

The cost of attention

Every head of every layer computes a $T \times T$ matrix of scores.

Context T	**Scores per head per layer**
512	262 144
1 024	1 048 576
2 048	4 194 304

Doubling the context quadruples the attention time and memory.

That is the price of letting any token read any token directly. Fused kernels (FlashAttention) cut the memory, not the T^2 compute. Measured in notebook 6.1.

Recurrence vs attention

Per layer, sequence of T tokens of width d (Vaswani et al., 2017):

	Recurrent layer	**Self-attention layer**
compute	$T\,d^2$	$T^2\,d$
sequential steps	T	1
longest path between two tokens	T	1

Attention wins on **parallelism** and **path length**: the two problems of Session 05.

It loses when T is much larger than d.

Part 2 · The Transformer block

1. Self-attention in full — heads, masks, position, cost
2. **The Transformer block** — attention plus everything from Session 02
3. Pretraining — tokens, next-token prediction, masked prediction
4. Scale and foundation models — text, images, time series, tables
5. Using a pretrained model — zero-shot, frozen, fine-tuned, LoRA

> Part 1: attention **mixes** tokens. By itself it is a weighted average, linear in V. It needs a nonlinearity, depth and a way to train that depth.

Two sublayers

$$\begin{aligned} X &\leftarrow X + \mathrm{MHA}\left(\mathrm{LN}(X)\right) && \text{mixes information } \textbf{across} \text{ tokens} \\ X &\leftarrow X + \mathrm{MLP}\left(\mathrm{LN}(X)\right) && \text{transforms } \textbf{each} \text{ token on its own} \end{aligned}$$

Attention is the only new piece.

Everything else is from Session 02: residual connections, LayerNorm, an MLP, GELU.

Pre-norm and the clean residual path

LayerNorm (Session 02): normalizes each token over its d features, not over the batch. It works the same in training and inference, and for any T.

Pre-norm: LN goes **inside** each branch, before the sublayer. The residual path from input to output is a pure sum, as promised in Session 02:

$$X_D = X_0 + \sum_{l=1}^{D} F_l(\cdot)$$

Post-norm (the original paper) puts LN **on** the residual path; deep stacks then need a careful warmup to train (Xiong et al., 2020).

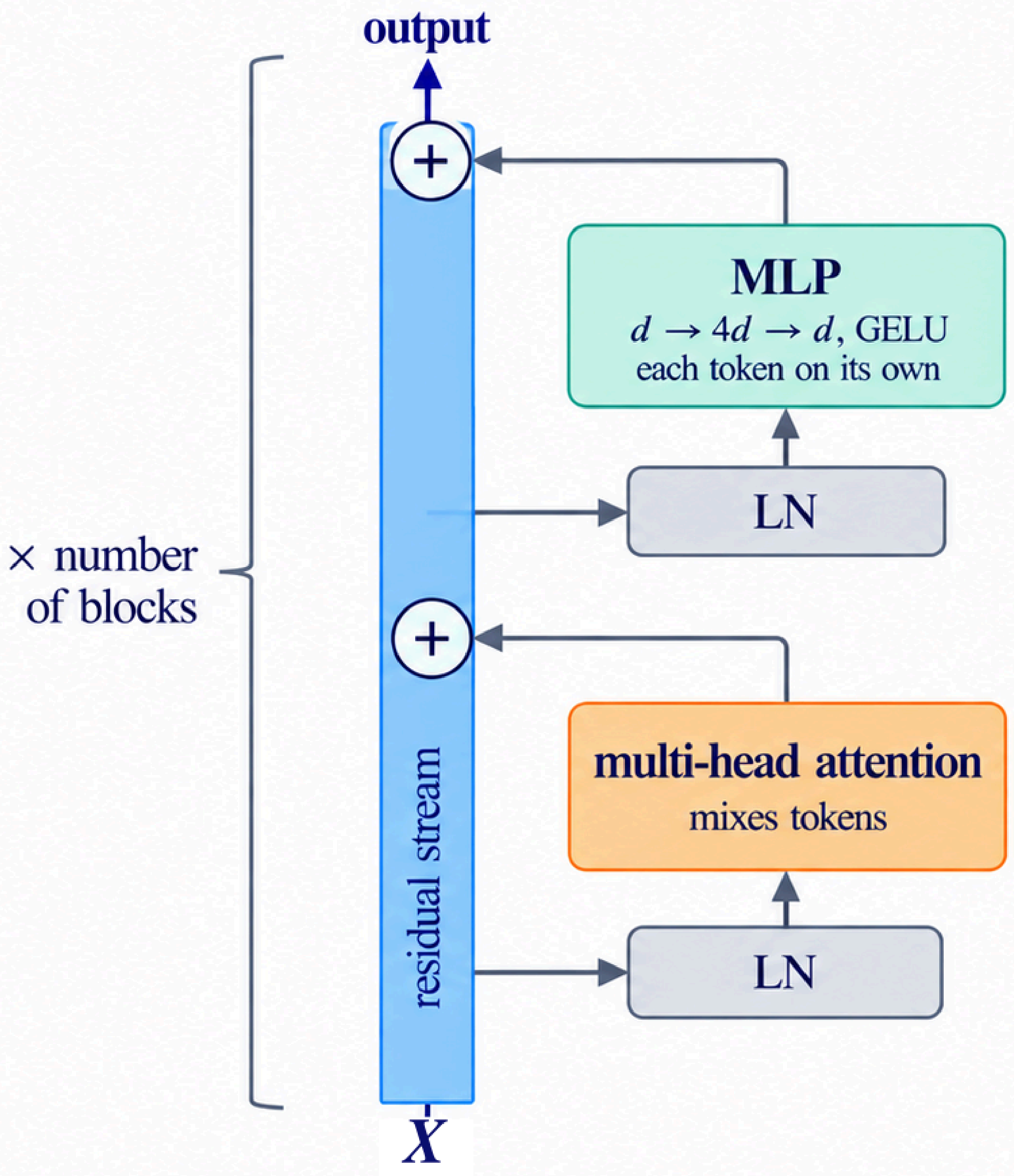
The MLP and GELU

$$\mathrm{MLP}(\mathbf{x}) = W_2\,\varphi(W_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2, \qquad W_1 \in \mathbb{R}^{4d\times d},\ W_2 \in \mathbb{R}^{d\times 4d},\ \varphi = \mathrm{GELU}$$

Applied to every token separately, with the **same** weights: a per-token network.

It holds $8d^2$ of the block's $12d^2$ parameters: **two thirds**.

GELU, from Session 02: $\varphi(z) = z\,\Phi(z)$.



output

× number of blocks

residual stream

MLP

$d \to 4d \to d$, GELU

each token on its own

LN

multi-head attention

mixes tokens

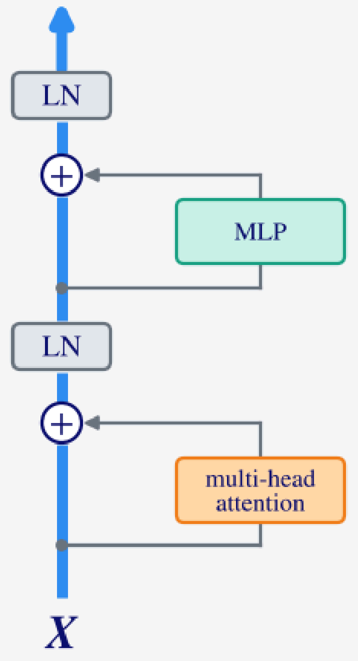
LN

X

$$X \leftarrow X + \mathrm{MLP}(\mathrm{LN}(X))$$

$$X \leftarrow X + \mathrm{MHA}(\mathrm{LN}(X))$$

post-norm (original)


LN
MLP
LN
multi-head attention
X

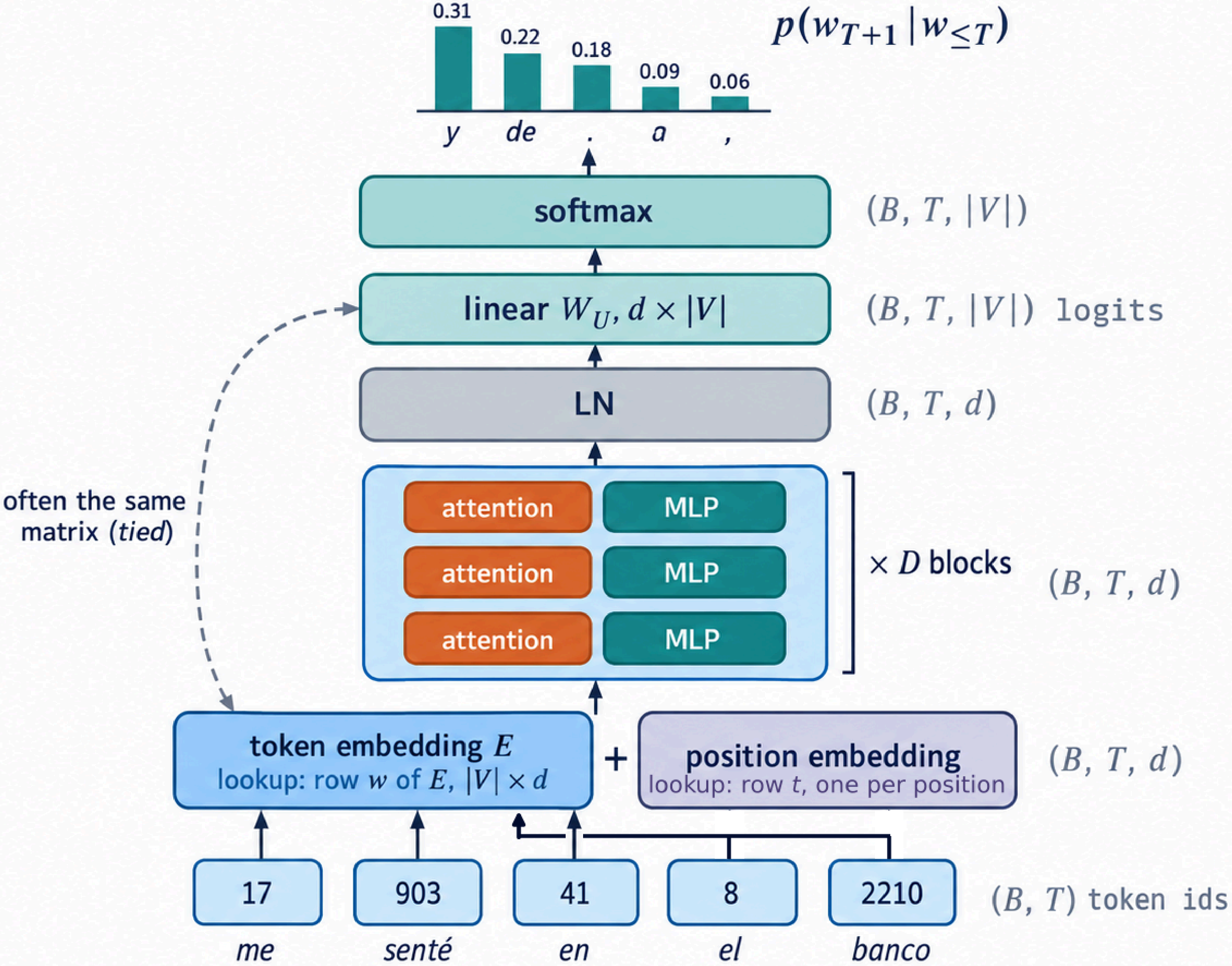
LN on the residual path

The block in motion

S06-A02 · One block

Tokens flow through one pre-norm block: LN → attention (tokens exchange information, arrows between columns) → add; LN → MLP (each column on its own) → add. The residual stream stays a straight band.

`../figures/s06_a02_block.mp4` · route: Manim (`escenas_s06.py`)


0.31
0.22
0.18
0.09
0.06
y
de
.
a
,
$p(w_{T+1} | w_{\le T})$
softmax
$(B, T, |V|)$
linear W_U, $d \times |V|$
$(B, T, |V|)$ logits
LN
(B, T, d)
often the same
matrix (tied)
attention
MLP
attention
MLP
attention
MLP
$\times D$ blocks
(B, T, d)
token embedding E
lookup: row w of E, $|V| \times d$
+
position embedding
lookup: row t, one per position
(B, T, d)
17
903
41
8
2210
(B, T) token ids
me
senté
en
el
banco

Counting parameters

Per block, ignoring biases and LayerNorm:

$$\underbrace{4d^2}_{W_Q, W_K, W_V, W_O} + \underbrace{8d^2}_{W_1, W_2} = 12d^2$$

GPT-2 small: $d = 768$, 12 blocks, vocabulary of 50 257, context of 1 024.

Part	Parameters
12 blocks: $12 \times 12d^2$	84.9 M
token embedding $\lvert V \rvert \cdot d$	38.6 M
position embedding $1024 \cdot d$	0.8 M
total	**≈ 124 M**

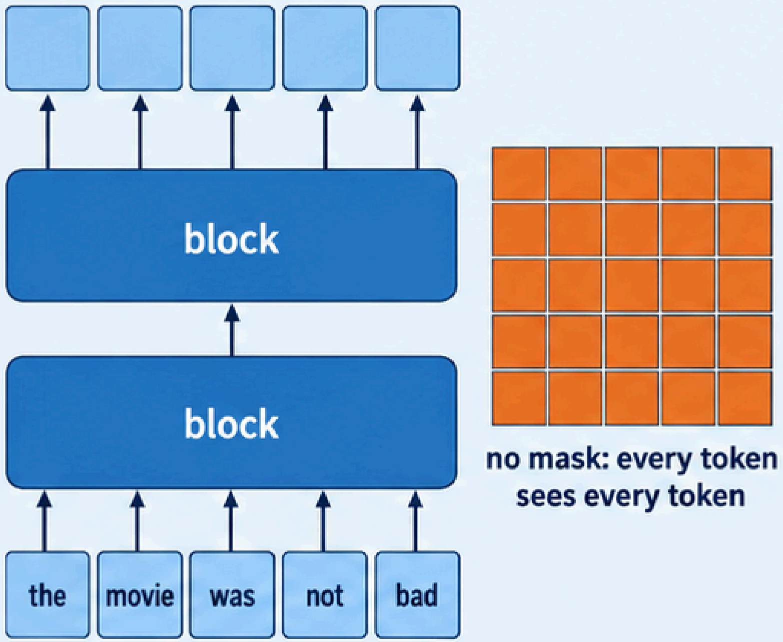
Notebook 6.2 checks the formula against `sum(p.numel() for p in model.parameters())`.

Three architectures

	Mask	Reads	Typical use	Examples
encoder	none (padding only)	the whole input	understand an input	BERT, ViT
decoder	causal	the past	generate, one token at a time	GPT
encoder-decoder	none in the encoder, causal in the decoder, plus **cross-attention**	the input and the output so far	input sequence → output sequence	original Transformer, T5

The encoder-decoder is the seq2seq of Session 05 without recurrence: cross-attention is the Bahdanau attention.

encoder

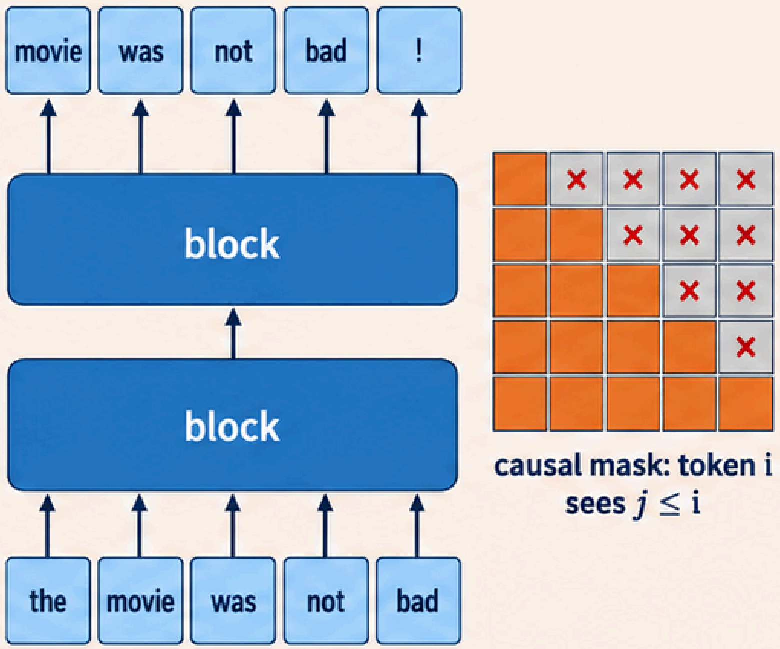

block
block
the
movie
was
not
bad

no mask: every token sees every token

understand an input

BERT, ViT

decoder

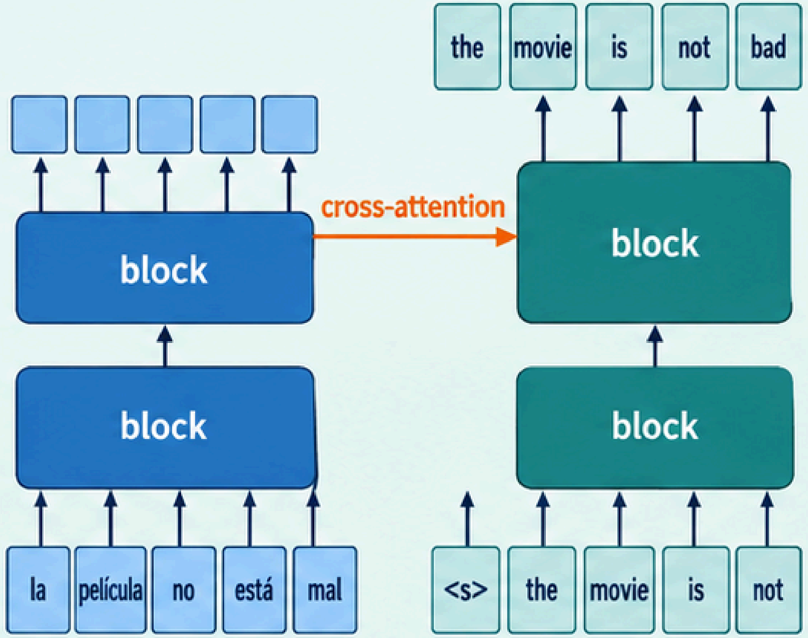

movie
was
not
bad
!
block
block
the
movie
was
not
bad

causal mask: token i sees $j \le i$

generate, one token at a time

GPT

encoder-decoder


the
movie
is
not
bad
cross-attention
block
block
block
block
la
película
no
está
mal
<s>
the
movie
is
not

input sequence → output sequence

original Transformer, T5

Prediction before experiment

The **dates task** of Session 05: Spanish sentences with a date in words → ISO date.

S05, notebook 5.5: GRU encoder-decoder with additive attention, $d = 24$: 100 % exact.

Now: a 2-layer Transformer encoder-decoder, $d = 64$.

- Which one takes less time per epoch?
- Which one reaches 100 % in fewer epochs?
- And the Transformer **without** positional embeddings?

Answer in notebook 6.2.

Training a Transformer

The Session 02 recipe, with two emphases:

- **AdamW**, learning-rate **warmup** then cosine decay;
- dropout on the attention weights and on each residual branch;
- gradient clipping (Session 05).

Warmup matters more than in a CNN: early Adam steps are large, and large updates to W_Q, W_K can push the scores into the saturated softmax of V13.

What the Transformer changed

- **No recurrence:** all positions in parallel during training.
- **Path length 1** between any two tokens.
- **One block** for text, images (ViT, Session 03), audio and time series: only the tokenizer changes.

What it costs:

- T^2 in time and memory;
- no built-in notion of **order** or **locality**: both must be added or learned from data.

Practice A — Attention and the block

Worked notebooks, one change per cell:

- 6.1 · Self-attention · CPU: five lines of attention checked against `F.scaled_dot_product_attention`; $\sqrt{d_k}$; heads; masks; permutation with and without position; time and memory vs T
- 6.2 · The Transformer block · CPU/GPU: the pre-norm block; parameters vs $12d^2$; the dates task of 5.5 with a Transformer; without position; post-norm without warmup

> Write down for 6.2: parameters, time per epoch, epochs to 100 %, for the S05 model and each Transformer.

Part 3 · Pretraining

1. Self-attention in full — heads, masks, position, cost
2. The Transformer block — attention plus everything from Session 02
3. **Pretraining** — tokens, next-token prediction, masked prediction
4. Scale and foundation models — text, images, time series, tables
5. Using a pretrained model — zero-shot, frozen, fine-tuned, LoRA

> Part 2: a block that trains in parallel and fits any sequence. Session 04: unlabeled data can **supervise itself**.

Subword tokens

Unit	Vocabulary	Sequence length	Unknown words
words (Session 05)	very large	short	`[UNK]`
characters	~100	long, and attention costs T^2	none
subwords (BPE)	chosen, e.g. 32 000	in between	none

Byte-pair encoding: start from bytes; count adjacent pairs in the training text; merge the most frequent pair into a new token; repeat until the vocabulary has the chosen size.

Frequent words end up as one token; rare words split into two or three pieces. Trained in notebook 6.3, where the actual splits of the novel are printed.

BPE on a Spanish sentence

S06-F10 · BPE

One Spanish sentence from characters to tokens through the first merges; vocabularies of 256, 2 000 and 8 000 tokens against the number of tokens per sentence.

`../figures/s06_f10_bpe.png` · route: Script (notebook 6.3)

Next-token prediction

$$J(\theta) = -\frac{1}{T-1}\sum_{t=1}^{T-1} \log p_\theta\big(w_{t+1} \mid w_1, \ldots, w_t\big)$$

At every position, a cross-entropy over the whole vocabulary: the classification loss of Session 01, with $|V|$ classes.

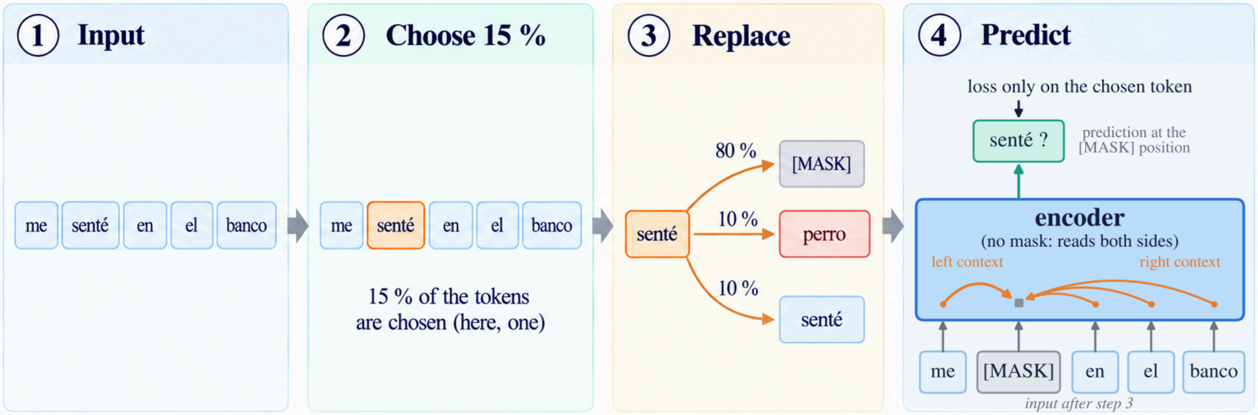
The labels are **the text itself, shifted by one**. Nobody annotates anything: self-supervision, as in Session 04.

Masked prediction for text

BERT (Devlin et al., 2019): hide 15 % of the tokens and predict them from **both** sides.

Session 04: images need about 75 % masked, text only 15 %. A word is much harder to guess from its neighbors than a pixel patch.

	Next token (GPT)	Masked (BERT)
architecture	decoder, causal mask	encoder, no mask
sees	the left context	both sides
good for	generating	representations for classification



① **Input**

me senté en el banco

② **Choose 15 %**

me senté en el banco

15 % of the tokens
are chosen (here, one)

③ **Replace**

senté

80 % [MASK]

10 % perro

10 % senté

④ **Predict**

loss only on the chosen token

senté ?

prediction at the
[MASK] position

encoder
(no mask: reads both sides)

left context

right context

me [MASK] en el banco

input after step 3

the model cannot tell which tokens were chosen, so it must keep a good representation of every token.

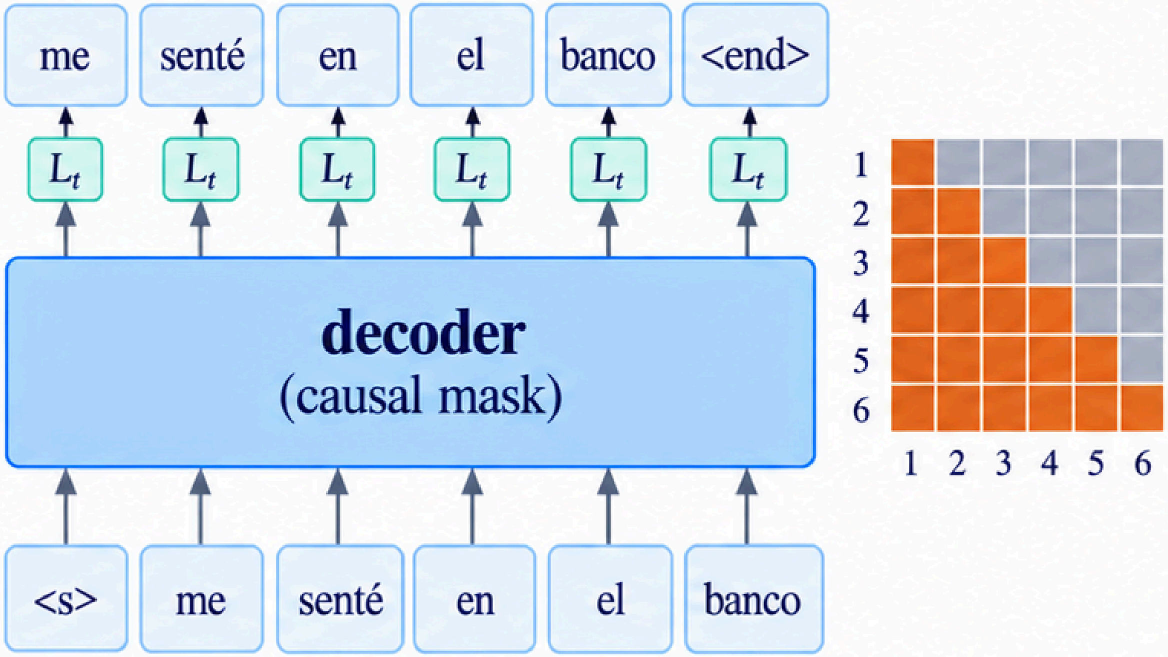
The causal mask makes training parallel

One forward pass over a sequence of T tokens gives T **predictions** and T **losses** at once, each one seeing only its own past.

```
logits = model(tokens)                              # (B, T, |V|), causal mask inside
loss = F.cross_entropy(
    logits[:, :-1].reshape(-1, V),                  # position t predicts...
    tokens[:, 1:].reshape(-1))                      # ...token t+1
```

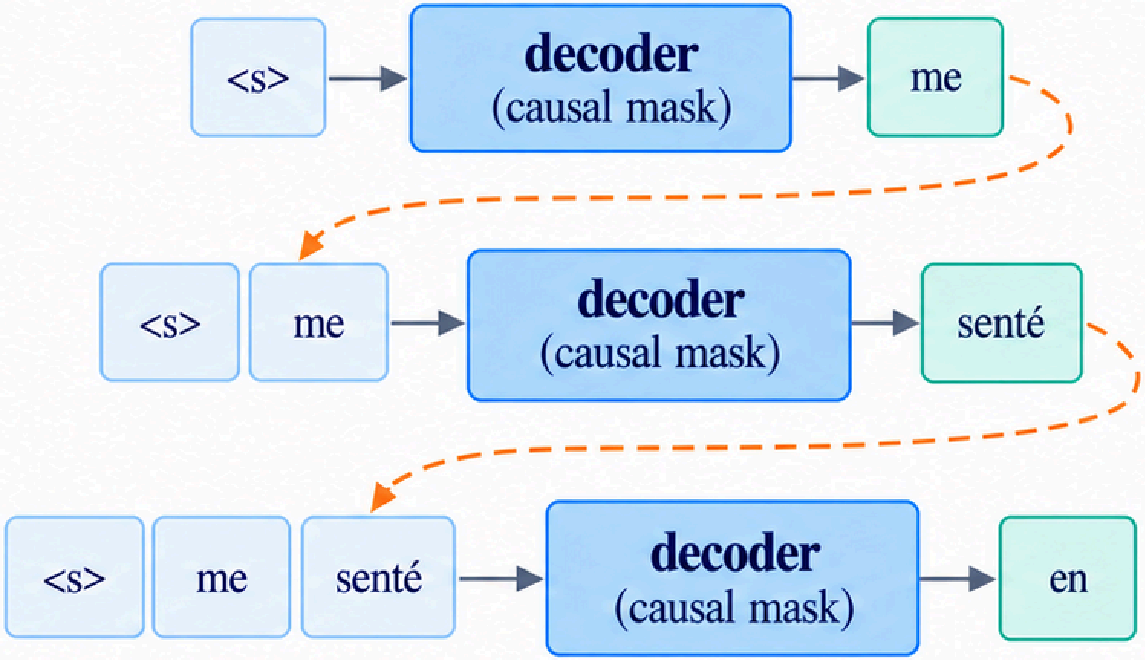
Training is parallel along T. **Generation** is still one token at a time: the next input is the token just produced.

training: one forward pass


me
senté
en
el
banco
<end>
L_t
L_t
L_t
L_t
L_t
L_t
decoder
(causal mask)
<s>
me
senté
en
el
banco
1
2
3
4
5
6
1 2 3 4 5 6

6 predictions and 6 losses at once

generation: one token per step


<s>
decoder
(causal mask)
me
<s>
me
decoder
(causal mask)
senté
<s>
me
senté
decoder
(causal mask)
en

the next input is the token just produced

Generating text

S06-A03 · Autoregressive generation

The prompt goes in; the logits of the last position become a distribution over the vocabulary; a token is sampled and appended; the window shifts. Then the same with low and high temperature.

`../figures/s06_a03_generation.mp4` · route: Manim (`escenas_s06.py`)

Sampling

From the logits $\mathbf{z}$ of the last position:

Rule	Next token
greedy	$\arg\max_w z_w$
temperature τ	sample from $\mathrm{softmax}(\mathbf{z}/\tau)$
top-k / top-p	sample only among the k most likely, or the smallest set with probability p

The temperature plays the same role as in InfoNCE (Session 04): small τ sharpens the distribution.

Low τ: safe and repetitive. High τ: varied and incoherent.

Prediction before experiment

A tiny GPT, about 2 M parameters, trained on a Spanish novel with the block of notebook 6.2.

Samples after **200 steps**, after **2 000 steps**, and from the long checkpoint.

Spelling, words, syntax, meaning: which one appears first, and which one never arrives at this size?

Answer in notebook 6.3.

A tiny GPT, learning

S06-F13 · Training loss and samples

Training loss of the tiny GPT with three samples printed at the steps where they were taken.

`../figures/s06_f13_gpt_samples.png` · route: Script (notebook 6.3)

What pretraining learns

To predict the next token well, a model has to learn, from the text alone:

- spelling and grammar;
- facts stated in the text;
- some patterns of reasoning that the text contains.

Nobody labels any of it.

GPT-2 (Radford et al., 2019): one model pretrained on web text did several tasks (summaries, translation, questions) **without** being trained for them.

Part 4 · Scale and foundation models

1. Self-attention in full — heads, masks, position, cost
2. The Transformer block — attention plus everything from Session 02
3. Pretraining — tokens, next-token prediction, masked prediction
4. **Scale and foundation models** — text, images, time series, tables
5. Using a pretrained model — zero-shot, frozen, fine-tuned, LoRA

> Part 3: next-token prediction turns any text into supervision. How much text, and how big a model?

Scaling laws

Test loss falls as a **power law** in the number of parameters N, the training tokens D and the compute C, over many orders of magnitude (Kaplan et al., 2020):

$$L(N) \approx \left(\frac{N_c}{N}\right)^{\alpha_N}, \qquad \alpha_N \approx 0.08$$

A straight line on log-log axes: **predictable** gains, with small exponents. Ten times more parameters lowers the loss by a constant factor, not to zero.

Loss against compute

S06-F14 · Scaling

Test loss against training compute on log-log axes, from the published power-law fits; optionally, points from the small sweep of the companion notebook.

`../figures/s06_f14_scaling.png` · route: Script (`make_s06_f07_scaling.py`)

Compute-optimal training

Training a Transformer costs about

$$C \approx 6\,N\,D \;\; \text{FLOPs}$$

(2 for the forward pass, 4 for the backward, per parameter and per token).

For a fixed budget C, grow N and D **together**: about **20 tokens per parameter** (Hoffmann et al., 2022).

Chinchilla, 70 B parameters on 1.4 T tokens, beat models four times larger trained on less data.

What is a foundation model?

> Trained on **broad data at scale**, usually with self-supervision, and then **adapted** to many downstream tasks (Bommasani et al., 2021).

- It is a **pattern of training and use**, not an architecture.
- A Transformer is not a foundation model by itself; a Transformer pretrained on the web and reused for a hundred tasks is.
- The ImageNet backbone of Session 03 was an early, small version of the same idea.

Beyond text: vision

CLIP (Radford et al., 2021): 400 M image–caption pairs, trained with InfoNCE (Session 04). The positive pair is an image and **its caption**.

Zero-shot classification: embed the image; embed one text per class, "a photo of a {class}"; pick the closest text. No training on the task.

DINO, MAE (Session 04): vision backbones pretrained without labels, used through probes or fine-tuning.

CLIP zero-shot on EuroSAT

S06-F15 · CLIP zero-shot

One EuroSAT image against the text embeddings of the 10 class prompts: cosine similarity as bars, the predicted class highlighted.

`../figures/s06_f15_clip_zeroshot.png` · route: Script (notebook 6.4)

Beyond text: time series and tables

Chronos (Ansari et al., 2024): scale each series, quantize its values into a fixed set of bins, treat the bins as tokens, and train a language model on many series (real and synthetic). Forecast by **sampling** future tokens: several possible futures, not one number.

TabPFN (Hollmann et al., 2023): a Transformer pretrained on millions of **synthetic** tables. For a new small table it predicts in one forward pass, with the training rows as context.

Both are used **zero-shot**: no gradient step on your data.

Zero-shot is a claim to check

A foundation model may never have seen data like yours:

- satellite bands that are not RGB;
- a hospital's bed occupancy, a factory's sensors;
- Spanish clinical notes.

The **baseline still has to be beaten** (Session 05: the commonsense forecast). A pretrained model that loses to "tomorrow equals today" is not useful, however large.

Prediction before experiment

EuroSAT, 10 classes

- CLIP zero-shot
- CLIP features + linear probe, 10 % of the labels
- SimCLR probe from Session 04, 10 % of the labels

Bike Sharing, 24 h ahead

- commonsense baseline
- LSTM from the Session 05 exercise
- Chronos zero-shot

Which one comes out ahead in each case? Answer in notebook 6.4.

The cost of size

Training: a frontier model needs compute only a few labs can pay for.

Inference memory: parameters × bytes per parameter.

Model	16-bit weights
124 M (GPT-2 small)	0.25 GB
7 B	14 GB
70 B	140 GB

Most practitioners **use** foundation models; they do not train them. So: how to use one?

Part 5 · Using a pretrained model

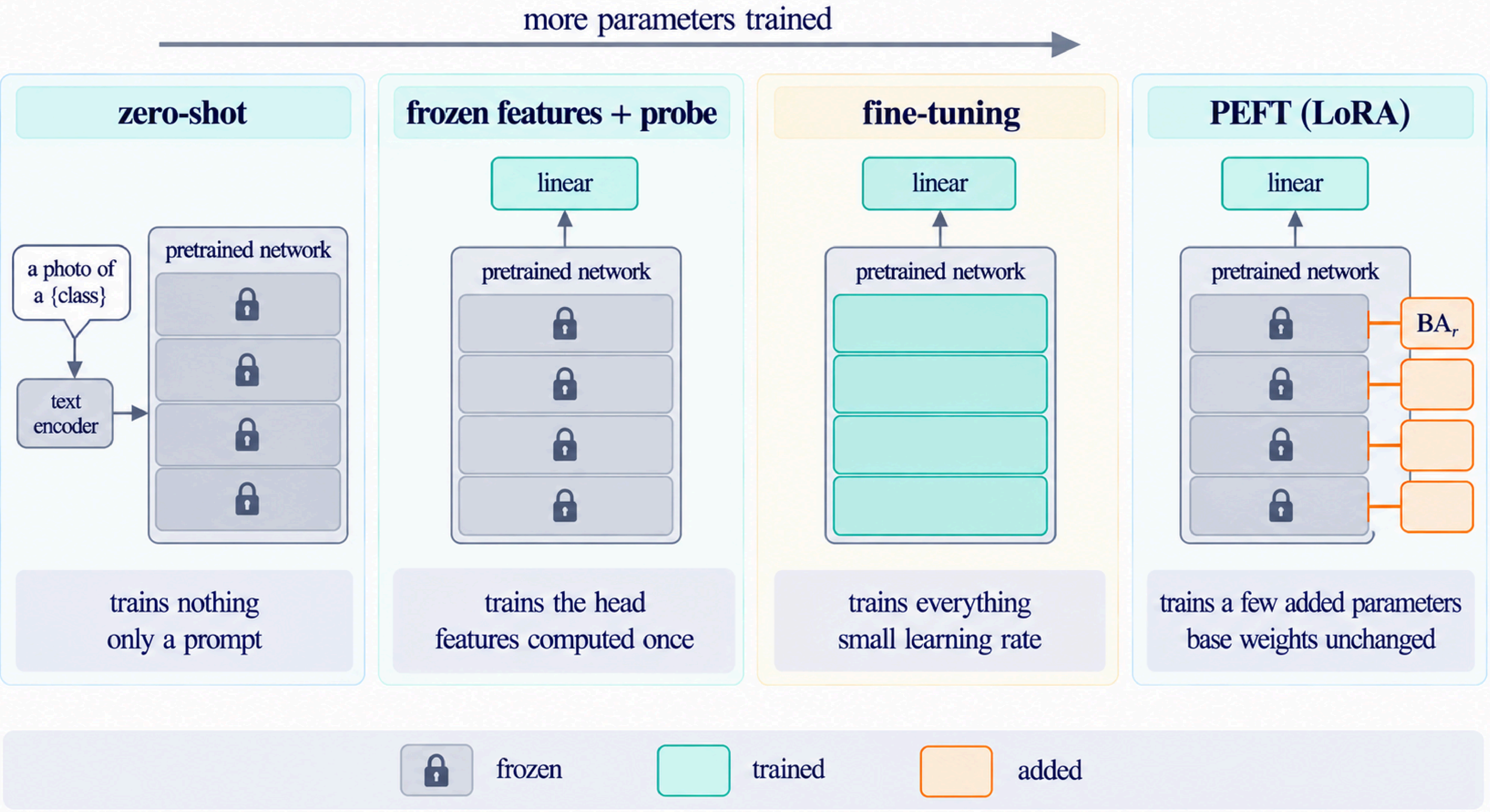
1. Self-attention in full — heads, masks, position, cost
2. The Transformer block — attention plus everything from Session 02
3. Pretraining — tokens, next-token prediction, masked prediction
4. Scale and foundation models — text, images, time series, tables
5. **Using a pretrained model** — zero-shot, frozen, fine-tuned, LoRA

> Part 4: pretrained models exist for text, images, series and tables. The question now is **how much of one to train**.

Four ways to use it

	Trains	Cost	When
zero-shot	nothing	one forward pass	no labels; check against a baseline
frozen features + probe	the head	seconds, features cached	few labels, domain close to pretraining
fine-tuning	everything	memory for all gradients	enough labels, or a distant domain
PEFT (LoRA)	a few added parameters	a fraction of fine-tuning	limited memory, many tasks

The three strategies of Sessions 03 and 04, plus one.



more parameters trained

zero-shot

a photo of a {class}
text encoder
pretrained network

trains nothing
only a prompt

frozen features + probe

linear
pretrained network

trains the head
features computed once

fine-tuning

linear
pretrained network

trains everything
small learning rate

PEFT (LoRA)

linear
pretrained network
BA_r

trains a few added parameters
base weights unchanged

🔒 frozen 🟩 trained 🟧 added

Frozen features

Run the encoder **once** over the data and cache $\mathbf{h}$:

- the output of the `[CLS]` token, or the mean of all token outputs (masking the padding);
- then train a linear layer on the cached vectors, in seconds.

```
with torch.no_grad():
    h = encoder(**batch).last_hidden_state[:, 0]   # [CLS]
```

Cheap and robust with few labels; limited by what pretraining put into $\mathbf{h}$ (Session 04: probes measure exactly that).

Fine-tuning

All weights trainable, with the Session 03 recipe:

- a **small learning rate**, 10^{-5} to $5 \cdot 10^{-5}$;
- warmup, so the random head does not wreck the pretrained weights;
- few epochs: the model starts close to a good solution.

The best choice with enough labels or a distant domain.

The cost: gradients and AdamW state for **every** weight, about 4× the memory of the weights alone, and a full copy of the model per task.

LoRA, in one slide

Freeze $W_0 \in \mathbb{R}^{d\times d}$. Learn a **low-rank** update:

$$W = W_0 + BA_r, \qquad B \in \mathbb{R}^{d\times r},\ A_r \in \mathbb{R}^{r\times d},\ r \ll d$$

- trainable values: $2dr$ instead of d^2; with $d = 768$, $r = 8$: **12 288 vs 589 824, 2 %**;
- B starts at zero, so training starts from the pretrained model exactly;
- after training, merge $W_0 + BA_r$: inference costs nothing extra.

(Hu et al., 2021.) A_r carries a subscript: A is the attention matrix.

LoRA, drawn

S06-F17 · From fine-tuning to LoRA, in three steps

① fine-tuning updates W with ΔW (d^2 values); ② the same written as two paths, $W_0\mathbf{x} + \Delta W\mathbf{x}$; ③ LoRA replaces ΔW with the thin factors BA_r ($2dr$ values).

`../figures/s06_f17_lora.png` · route: Generator (`figure_prompts.md`)

From a language model to an assistant

A pretrained language model **continues** text. Ask it a question and it may write three more questions.

Three stages:

1. **Pretraining** (Part 3): next token on a huge corpus.
2. **Supervised fine-tuning (SFT):** next token on (question, answer) pairs.
3. **Preference tuning** (RLHF, DPO): learn from which of two answers people prefer. One line today.

A **chat template** turns a pair into one sequence of tokens with role markers:

```
<user> ¿Qué es un átomo? <assistant> Partícula mínima <end>
```

The integrating exercise is stage 2 in miniature: teach a style.

The next-token loss, token by token

`<user>` ¿Qué es un átomo? `<assistant>` Partícula mínima `<end>`

t	**target** w_{t+1}	$p(\text{target})$	L_t
1	¿Qué	0.20	1.61
2	es	0.60	0.51
3	un	0.50	0.69
4	átomo?	0.01	4.61
5	`<assistant>`	0.98	0.02
6	Partícula	0.05	3.00
7	mínima	0.10	2.30
8	`<end>`	0.30	1.20

Logits $Z \in \mathbb{R}^{T\times|V|}$. Position t reads tokens $1, \ldots, t$ and predicts token $t+1$:

$$L_t = -\log \operatorname{softmax}(\mathbf{z}_t)_{w_{t+1}}$$

A cross-entropy over the vocabulary at every position (Session 01).

9 tokens give 8 predictions: the last one has nothing to predict.

The loss in motion

S06-A04 · The masked next-token loss

A short chat example as a row of tokens. One distribution per position, shifted one step to face the next token; the probability of the true token becomes a bar L_t. The mask turns off the template and question bars, and the mean is taken over the answer. Last, the same sequence after LoRA: the answer bars shrink.

`../figures/s06_a04_masked_loss.mp4` · route: Manim (`escenas_s06.py`)

Loss only on the answer

Mask $m_t = 1$ if the target w_{t+1} belongs to the answer (including `<end>`, so the model learns to stop), 0 otherwise:

$$J = \frac{\sum_t m_t L_t}{\sum_t m_t}$$

Without the mask: $J = 13.94/8 = 1.74$.

The largest term is "átomo?" (4.61): the model would learn to **predict the user's question**. The template (0.02) dilutes the rest.

With the mask:
$J = (3.00 + 2.30 + 1.20)/3 = 2.17$.

Only the answer counts.

```
L = F.cross_entropy(logits[:, :-1].reshape(-1, V), tokens[:, 1:].reshape(-1), reduction="none")
J = L[mask[:, 1:].reshape(-1)].mean()      # one L_t per position, mean over the answer only
```

Per-token loss of a real model

S06-F18 · The masked loss

One chat example as tokens colored by role (template, question, answer), with the per-token loss L_t of the base model before fine-tuning; masked positions greyed; the masked mean against the unmasked mean.

`../figures/s06_f18_masked_loss.png` · route: Script (notebook 6.6)

Measuring a style

Held-out NLL: the same masked J on test pairs, with no gradient. Lower is better.

$\exp(J)$ is the **perplexity**: roughly, among how many tokens the model is hesitating at each step. In the example, $e^{2.17} \approx 8.7$.

Compare three conditions on the same test questions:

1. the base model;
2. the base model, with the style asked for in the prompt;
3. the base model with LoRA.

Only compare NLL between models with the **same tokenizer**: with other tokens, the sum runs over other units.

An LLM acting as judge is easier to read and harder to trust: optional.

Comparing fairly

- The same data split, the same number of steps or the same compute.
- Several seeds where they are cheap.
- Next to accuracy: **trainable parameters**, training time and peak memory.

The data contract for text: the **tokenizer** and the **maximum length** are part of the model, and travel with it.

Prediction before experiment

IMDB with **2 000** training reviews.

Model	Accuracy?	Trainable parameters?
LSTM (Session 05)		
DistilBERT frozen + probe		
DistilBERT fine-tuned		
DistilBERT + LoRA		

Rank them by accuracy. Rank them by trainable parameters. Do the two rankings agree?

Answer in notebook 6.5.

When to use what

Situation	First try
no labels at all	zero-shot, checked against a baseline
few labels, domain close to pretraining	frozen features + probe
enough labels, or a distant domain	fine-tuning
limited memory, or many tasks on one base	LoRA

Always with a **baseline** in the first row of the table.

Practice B — Pretraining and adaptation

In class, one change per cell, GPU:

- 6.3 · Pretraining a tiny GPT: a BPE tokenizer; a GPT with the block of 6.2; a few minutes of training, then the long checkpoint; samples; sampling rules
- 6.5 · Adapting a pretrained model: IMDB with DistilBERT frozen, fine-tuned and with LoRA; accuracy, trainable parameters, time and memory

At home:

- 6.4 · Foundation models: CLIP on EuroSAT, Chronos on Bike Sharing

> Write down for 6.5 one row per model: accuracy, trainable parameters, time, peak memory.

Integrating exercise — Does it need fine-tuning?

6.6 · Integrating exercise · a pretrained LLM that must answer in a given style

- the chat template and the **mask** m_t that keeps the loss on the answer;
- the masked loss of the slides, checked against the library's;
- LoRA under a fixed budget, logged to **MLflow**; held-out NLL for base, base with the style in the prompt, and LoRA;
- `responder_raw`: an answer from a raw question that respects the contract.

> Is asking for the style in the prompt enough, or does the model have to be fine-tuned?

Adapted from MIT 6.S191, Lab 3 (MIT License).

The course in six questions

Session	**Question**	**What it added to the loop** $x \to f_\theta(x) \to L \to \nabla_\theta L \to \theta'$
01	How does a network learn?	the loop itself: loss, gradient, update
02	Why can we train deep networks?	initialization, normalization, residuals, optimizers
03	How does data structure shape the network?	architecture as an assumption: convolution, patches
04	Can a network learn without labels?	the objective chooses the representation
05	How does a network remember the past?	recurrence, gates, attention over time
06	What if we keep the attention and drop the recurrence?	pretraining at scale, then adapting

What each choice assumes

	Assumes	**Pays with**
self-attention	any token may matter for any other	T^2 time and memory
no position	order does not matter	everything order carries
pretraining	the pretraining data resembles yours	errors where it does not
zero-shot	the task is already in the model	no way to correct it without labels
frozen features	the representation already fits the task	the gap a probe cannot close
LoRA	the needed update is low rank	a little accuracy, sometimes

Summary

S06-F19 · Accuracy against trainable parameters

IMDB: the Session 05 models (bag of embeddings, 1-D CNN, LSTM) and the four strategies of notebook 6.5, accuracy against trainable parameters on a log axis.

`../figures/s06_f19_summary.png` · route: Script (notebook 6.5)

Exit question

A clinic has **800 labeled Spanish clinical notes** (3 classes) and **one 16 GB GPU**.

Choose two and justify:

- an LSTM trained from scratch;
- a multilingual encoder, frozen, plus a probe;
- full fine-tuning of the same encoder;
- LoRA on a 7 B decoder;
- ask a chat LLM zero-shot, and compare it with a baseline on a held-out set.